

TECNOLÓGICO DE MONTERREY

Escuela de Ingeniería y Ciencias

MA2006B · Álgebras Modernas para Criptografía

Casa Monarca

Gestor de Identidades

Manual Técnico

A00839082 - Paola Michelle Martínez Galeazzi

A00838589 - Steffany Mishell Lara Muy

A01424357 - Patricio Mourra Cossio

A00838606 - Santiago Dueñas Sanchez

A00840131 - María Paula Recinos Rios

A01198777 - Andrés Alarcón Navarro

Grupo 603 · Equipo 3

Etapa 1 — Sistema de gestión de identidades para personas migrantes atendidas por Casa Monarca A.C.

Monterrey, N.L. · 2026

Licencia MIT · Código fuente de uso público

Licencia MIT

Copyright (c) 2026 Equipo 3 — Grupo 603, Tecnológico de Monterrey.

Se concede permiso, de forma gratuita, a cualquier persona que obtenga una copia de este software y los archivos de documentación asociados, para usar el software sin restricción, incluyendo sin limitación los derechos de usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del software, y para permitir a las personas a quienes se proporcione el software hacer lo mismo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sustanciales del software.

EL SOFTWARE SE PROPORCIONA «TAL CUAL», SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA. EN NINGÚN CASO LOS AUTORES O TITULARES DEL COPYRIGHT SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑO U OTRA RESPONSABILIDAD DERIVADA DEL USO DEL SOFTWARE.

Uso institucional. Este documento fue elaborado como entregable académico para el reto Casa Monarca, en el marco de la materia MA2006B del Tecnológico de Monterrey. Describe un sistema funcional desplegado para uso real por parte de Casa Monarca A.C.

1. Introducción

1.1 Contexto

Casa Monarca A.C. es una organización sin fines de lucro con sede en Monterrey, Nuevo León, que brinda atención integral a personas migrantes en tránsito o en proceso de regularización migratoria. La información que maneja incluye nombres completos, países de origen, grupos de vulnerabilidad y redes de contacto, una filtración de estos datos no representa únicamente un riesgo legal, sino un riesgo directo para la seguridad física de los involucrados.

El sistema de Gestión de identidades de este manual fue desarrollado para combatir la ausencia de controles y seguridad criptográfica, además de la falta de registros de auditoría verificables en los procesos de la organización.

1.2 Alcance

El sistema abarca los siguientes módulos funcionales:

- Autenticación con dos factores: contraseña personal más certificado criptográfico PKCS#12 para los roles con capacidad de modificación de datos.
- Gestión de colaboradores: alta, edición, desactivación y restablecimiento de contraseña.
- Infraestructura de clave pública (PKI) interna: emisión, entrega, revocación y renovación de certificados X.509.
- Registro de migrantes con flujo de aprobación multi-firma (Voluntario → Operativo → Coordinador/Administrador).
- Derechos ARCO completos conforme a la LFPDPPP.
- Bitácora de auditoría con más de treinta tipos de evento, exportable a Excel.
- Panel de administración con estadísticas en tiempo real.

1.3 Stack tecnológico

Componente	Tecnología	Versión mín.
Backend	Python 3.12 + Flask	3.12 / 3.x
Base de datos	MySQL vía PyMySQL	8.0
Sesiones	Flask-Session (filesystem)	0.5.x
Criptografía	cryptography (PyCA)	41.x
Hashing contraseñas	Werkzeug / PBKDF2-HMAC-SHA256	3.x
Exportación Excel	openpyxl	3.x
Frontend	HTML5 + CSS3 + JS vanilla	—
Despliegue	Railway (PaaS)	—

1.4 Dimensión del proyecto

El archivo principal app.py contiene 4 224 líneas de código, 74 rutas Flask y 120 funciones definidas. Las vistas del sistema se distribuyen en seis plantillas HTML principales.

1.5 Estructura de directorios

```
casa-monarca/  
  app.py          ← Rutas, middleware y lógica de negocio  
  emitir_cert_admin.py ← Bootstrap del administrador inicial  
  README.md  
  ca/             ← Generado al primer arranque  
    ca_cert.pem  
    ca_key.pem  
    server.pem  
    server.key  
  db/  
    schema.sql  
    schema_mysql.sql  
    casa_monarca.db  
  flask_sessions_demo/  
  static/  
    css/admin.css  
    img/logo.png  
  templates/  
    admin.html  
    login.html  
    login_usb.html  
    cambiar_pwd.html  
    entregar_cert.html  
    panel_usuario.html  
    voluntario.html
```

2. Arquitectura del Sistema

2.1 Patrón general

El backend sigue un patrón MVC simplificado sin ORM. No se usa SQLAlchemy ni ninguna capa de abstracción adicional: todas las consultas son SQL puro mediante PyMySQL, lo que otorga control total sobre las transacciones y previene inyección SQL mediante el uso consistente de parámetros %s.

- Modelo: consultas SQL directas a través de la función auxiliar `_exec(db, sql, params)`.
- Vista: plantillas Jinja2 para páginas HTML y `jsonify()` para las APIs REST.
- Controlador: rutas Flask decoradas con `@verificar_certificado`.

2.2 Conexión a la base de datos

Se utiliza el objeto `g` de Flask para mantener una conexión por petición (`connection-per-request`):

```
def obtener_db():
    db = getattr(g, '_db', None)
    if db is None:
        g._db = pymysql.connect(
            host=DB_HOST, port=DB_PORT,
            user=DB_USER, password=DB_PASSWORD,
            database=DB_NAME,
            cursorclass=pymysql.cursors.DictCursor,
            autocommit=False,
            init_command="SET time_zone = 'America/Monterrey'"
        )
    return g._db
```

`autocommit=False` exige `commit()` explícito en cada operación de escritura, lo que garantiza coherencia transaccional. `DictCursor` devuelve las filas como diccionarios, simplificando la serialización a JSON. La conexión se cierra mediante el decorador `@app.teardown_appcontext`.

2.3 Inicialización de la base de datos

La función `inicializar_db()` se ejecuta automáticamente al arrancar el servidor. Su responsabilidad es triple:

1. Ejecutar `db/schema_mysql.sql` con sentencias `CREATE TABLE IF NOT EXISTS`.
2. Aplicar migraciones incrementales mediante `ALTER TABLE` envueltos en bloques `try/except`: si la columna ya existe, el error se ignora. Este enfoque es simple y efectivo para la escala del proyecto.
3. Sembrar usuarios y certificados iniciales si la tabla de usuarios está vacía.

2.4 Sistema de sesiones

Se usa `flask-session` con backend en sistema de archivos. `SESSION_USE_SIGNER=True` firma criptográficamente el identificador de sesión con `app.secret_key`, previniendo la falsificación de cookies.

La sesión almacena, entre otros campos:

- `session['rol']`, `session['usuario']`, `session['nombre']`: identidad del usuario en sesión.
- `session['cert_private_key']`: clave privada EC P-256 extraída del `.p12` al hacer login (solo para admin y coordinador; vive únicamente en memoria y se destruye al cerrar sesión).

- `session['pending_user']`: email en espera durante el flujo de autenticación de dos factores.

Sesiones en Railway: el filesystem de Railway se reinicia con cada despliegue, por lo que las sesiones activas se pierden al redespregar. Para un entorno de producción de alta disponibilidad se recomendaría Redis como backend de sesiones.

2.5 Decorador de autenticación

Todas las rutas protegidas utilizan `@verificar_certificado`, que valida la presencia del rol en sesión y carga los datos del usuario en el contexto de la petición:

```
def verificar_certificado(f):
    @wraps(f)
    def decorado(*args, **kwargs):
        if 'rol' not in session:
            return redirect(url_for('login'))
        g.rol = session['rol']
        g.usuario = session.get('usuario', '')
        g.nombre = session.get('nombre', 'Usuario')
        return f(*args, **kwargs)
    return decorado
```

Las rutas del administrador realizan además una verificación explícita: si `g.rol != 'admin'`, retornan HTTP 403.

2.6 Zona horaria

El sistema utiliza un offset fijo de UTC−6 en lugar de `zoneinfo` porque Railway no garantiza la disponibilidad de los archivos de zona horaria del sistema operativo. Se produce un objeto `datetime` naive (sin información de zona) compatible con el tipo `DATETIME` de MySQL.

Limitación conocida: el offset UTC−6 es fijo y no contempla el horario de verano. Monterrey observa CDT (UTC−5) aproximadamente de marzo a noviembre. En una versión futura se recomienda resolver la zona horaria con `pytz` o gestionar las marcas de tiempo en UTC en la base de datos y convertir al presentar en la interfaz.

2.7 Arquitectura del frontend

El frontend no utiliza ningún framework de JavaScript. Simula una Single Page Application mediante un patrón de visibilidad CSS: todas las secciones existen en el DOM desde el inicio y se muestran u ocultan cambiando la clase `.activa`. Toda comunicación con el servidor ocurre mediante `fetch()`, y el backend responde con JSON en todas sus APIs (`/admin/api/*`, `/api/*`).

3. Base de Datos

3.1 Tablas del sistema

Tabla	Propósito
usuarios	Cuentas de colaboradores: roles, hashes de contraseña, claves EC, contador de intentos fallidos.
certificados	Registro de todos los certificados X.509 emitidos: número de serie, estado (vigente / revocado / expirado), fechas.
certificados_revocados	Historial de revocaciones con motivo y responsable.
log_certificados	Bitácora de auditoría de todos los eventos del sistema.
migrantes	Padrón de personas atendidas con campo firmas_aprobaion (JSON con las firmas ECDSA del flujo de aprobación).
solicitudes_registro_migrante	Tickets de alta en proceso de aprobación multi-firma.
solicitudes Eliminacion	Solicitudes de eliminación con firma EC del coordinador.
solicitudes_arco_rect	Solicitudes de Rectificación ARCO (operativo → coordinador).
solicitudes_cancelacion_op	Solicitudes de Cancelación iniciadas por el operativo.
historial_eliminaciones	Archivo permanente de migrantes eliminados.
solicitudes_pwd	Solicitudes de restablecimiento de contraseña.

3.2 Tabla usuarios

Campo	Tipo	Descripción
id	INT PK AUTO	Identificador interno
usuario	VARCHAR UNIQUE	Email del colaborador (clave de acceso)
nombre	VARCHAR	Nombre completo
rol	ENUM	admin coord op voluntario
password_hash	TEXT	Hash PBKDF2-HMAC-SHA256 (Werkzeug)
debe_cambiar_pwd	TINYINT(1)	Forzar cambio en el primer inicio de sesión
serial_cert	VARCHAR	Serie del último certificado activo
activo	TINYINT(1)	Cuenta habilitada
bloqueado	TINYINT(1)	Bloqueado por intentos fallidos
intentos_fallidos	INT	Contador de intentos incorrectos
ec_private_key	TEXT	Clave privada ECDSA P-256 en PEM (solo para operativos)
ec_public_key	TEXT	Clave pública ECDSA P-256 en PEM

Almacenamiento de claves privadas: para los roles op y voluntario, la clave privada EC se almacena en la base de datos sin cifrado adicional. Para los roles admin y coord, la clave privada vive exclusivamente dentro del .p12 y se carga en memoria de sesión al hacer login; nunca se persiste en la base de datos ni en disco del servidor.

3.3 Tabla certificados

Campo	Tipo	Descripción
serial	VARCHAR UNIQUE	Número de serie hexadecimal
usuario	VARCHAR	Email del titular
nombre	VARCHAR	Nombre legible del titular
rol	ENUM	Rol al momento de la emisión
fecha_emision	DATE	Fecha de emisión
fecha_expiracion	DATETIME	Fecha de expiración
estado	ENUM	vigente revocado expirado
emitido_por	VARCHAR	Email del administrador que lo emitió

3.4 Tabla migrantes

El campo firmas_aprobacion contiene un objeto JSON con las firmas ECDSA de todos los actores que intervinieron en el registro:

```
{
  "op": {
    "usuario": "op@casamonarca.org",
    "mensaje": "REGISTRO_OP|12|Nombre|...",
    "firma": "<base64 DER>",
    "pubkey": "<PEM clave publica EC>"
  },
  "coord": {
    "usuario": "coord@casamonarca.org",
    "mensaje": "REGISTRO_COORD|12|Nombre|...|ancla_firma_op",
    "firma": "<base64 DER>",
    "pubkey": "<PEM clave publica EC>"
  }
}
```

3.5 Folio de registro

El folio se asigna inmediatamente tras la inserción en la tabla migrantes con el formato:

MIG-YYYYMM-NNNN (ejemplo: MIG-202604-0023)

3.6 Schema SQL de referencia

```
CREATE TABLE IF NOT EXISTS usuarios (  
  id    INTEGER PRIMARY KEY AUTOINCREMENT,  
  usuario TEXT NOT NULL UNIQUE,  
  nombre TEXT NOT NULL,  
  rol   TEXT NOT NULL CHECK (rol IN ('admin','coord','op','voluntario')),  
  password_hash TEXT NOT NULL,  
  debe_cambiar_pwd INTEGER NOT NULL DEFAULT 1,  
  serial_cert TEXT,  
  activo INTEGER NOT NULL DEFAULT 1,  
  intentos_fallidos INTEGER NOT NULL DEFAULT 0,  
  bloqueado INTEGER NOT NULL DEFAULT 0,  
  ec_private_key TEXT,  
  ec_public_key TEXT,  
  creado TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS certificados (  
  id    INTEGER PRIMARY KEY AUTOINCREMENT,  
  serial TEXT NOT NULL UNIQUE,  
  usuario TEXT NOT NULL,  
  rol   TEXT NOT NULL CHECK (rol IN ('admin','coord','op')),  
  fecha_emision TEXT NOT NULL,  
  fecha_expiracion TEXT,  
  estado TEXT NOT NULL DEFAULT 'vigente'  
    CHECK (estado IN ('vigente','revocado','expirado')),  
  emitido_por TEXT  
);  
  
CREATE TABLE IF NOT EXISTS log_certificados (  
  id    INTEGER PRIMARY KEY AUTOINCREMENT,  
  serial TEXT,  
  usuario TEXT,  
  rol TEXT,  
  ip_origen TEXT,  
  resultado TEXT NOT NULL,  
  detalle TEXT,  
  fecha TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

4. Infraestructura de Clave Pública (PKI)

4.1 Visión general

El sistema implementa una PKI interna de un solo nivel, una CA raíz que firma directamente los certificados de los colaboradores. No existe una CA intermedia. La CA utiliza RSA-4096 para firmar los certificados, mientras que las operaciones del sistema en tiempo de ejecución, las aprobaciones, cancelaciones, solicitudes ARCO, se firman con curvas elípticas P-256. Esto es intencional y se explica en detalle en la sección 4.4.

4.2 Autoridad Certificadora

Al primer arranque, inicializar_ca() genera:

- Una clave privada RSA-4096 para la CA.
- Un certificado autofirmado con BasicConstraints(ca=True) y diez años de validez.

Ambos artefactos se mantienen en ca/ca_cert.pem y ca/ca_key.pem. La clave privada de la CA se cifra con BestAvailableEncryption(CA_PASSWORD).

```
ca_key = rsa.generate_private_key(public_exponent=65537, key_size=4096)
ca_cert = (
    x509.CertificateBuilder()
    .subject_name(ca_name)
    .issuer_name(ca_name)          # auto-firmado
    .public_key(ca_key.public_key())
    .serial_number(x509.random_serial_number())
    .not_valid_before(datetime.utcnow() - timedelta(days=1))
    .not_valid_after(datetime.utcnow() + timedelta(days=3650))
    .add_extension(
        x509.BasicConstraints(ca=True, path_length=None), critical=True
    )
    .sign(ca_key, hashes.SHA256())
)
```

4.3 Emisión de certificados de usuario

La función generar_p12_real(email, nombre, rol, password) produce un certificado X.509 para cada colaborador con rol admin o coord. El certificado contiene una clave pública EC P-256 y es firmado por la CA (RSA).

```
private_key = ec.generate_private_key(ec.SECP256R1())

cert = (
    x509.CertificateBuilder()
    .subject_name(subject)
    .issuer_name(ca_cert.subject)      # firmado por la CA RSA
    .public_key(private_key.public_key()) # clave pública EC P-256
    .serial_number(x509.random_serial_number())
    .not_valid_before(datetime.utcnow() - timedelta(minutes=1))
    .not_valid_after(datetime.utcnow() + timedelta(days=30))
    .add_extension(
        x509.SubjectAlternativeName([x509.RFC822Name(email)]),
        critical=False
    )
    .sign(ca_key, hashes.SHA256())      # firma RSA-SHA256 de la CA
)

p12_bytes = pkcs12.serialize_key_and_certificates(
    name=nombre.encode('utf-8'),
```

```
key=private_key,          # clave privada EC P-256 del usuario
cert=cert,
cas=[ca_cert],
encryption_algorithm=BestAvailableEncryption(password.encode())
)
```

El campo SubjectAlternativeName con RFC822Name(email) es el elemento principal para la verificación de identidad en el login USB: el sistema exige que el email del certificado coincida exactamente con el usuario que intenta iniciar sesión.

4.4 RSA en la CA y EC en las operaciones

El sistema presenta dos usos criptográficos diferenciados:

La CA firma certificados con RSA-4096. Cuando el administrador emite un .p12 para un coordinador, la CA toma el certificado recién creado y lo firma con su clave privada RSA. Esa firma garantiza la autenticidad del certificado ante cualquier parte que confíe en la CA de Casa Monarca. El certificado resultante contiene la clave pública EC P-256 del coordinador, pero la firma que lo avala es RSA.

El coordinador firma operaciones con EC P-256. Una vez en sesión, cuando el coordinador aprueba una solicitud o firma una cancelación, usa su clave privada EC P-256 (extraída del .p12 y cargada en memoria de sesión). Esta operación no involucra a la CA en ningún punto.

Los dos usos nunca se mezclan ni se comparan entre sí, es decir, la CA solo firma certificados y los colaboradores solo firman mensajes de operación.

Esta diferenciación existe porque RSA-4096 ofrece mayor compatibilidad histórica y es la elección conservadora estándar para CAs raíz de larga duración. Esta arquitectura es similar a la empleada por muchas CAs históricamente por razones de compatibilidad y longevidad.

4.5 Validación del certificado en el login

```
private_key, certificate, _ = pkcs12.load_key_and_certificates(
    data, p12_password.encode("utf-8")
)
cert_email = extraer_email_de_cert(certificate)
if not cert_email or cert_email.lower() != email.lower():
    # El certificado no pertenece al usuario que intenta acceder
    return render_template('login_usb.html', error='...')
```

Tras validar la identidad, se verifica el estado del número de serie en la tabla certificados: si el certificado está revocado o expirado, el acceso se deniega aunque el archivo .p12 sea criptográficamente válido.

4.6 Ciclo de vida de la clave privada del .p12

```
Login exitoso con .p12
-> pkcs12.load_key_and_certificates(data, password)
-> session['cert_private_key'] = llave.private_bytes(PEM, PKCS8)

[Durante la sesión: cada firma usa session.get('cert_private_key')]

-> Logout / expiración de sesión
-> session.clear()
    La clave desaparece de memoria; nunca tocó el disco del servidor
```

5. Criptografía de Curva Elíptica (ECDSA P-256)

5.1 Fundamento matemático

ECDSA (Elliptic Curve Digital Signature Algorithm) opera sobre un grupo cíclico de puntos de una curva elíptica definida sobre un campo finito. La curva P-256 (SECP256R1 / prime256v1), estandarizada por NIST, se define sobre F_p mediante:

$$y^2 \equiv x^3 - 3x + b \pmod{p}$$

donde p es un primo de 256 bits y b es una constante del estándar.

El par de claves se genera eligiendo una clave privada $d \leftarrow R[1, n-1]$ y calculando el punto público $Q = d \cdot G$, donde G es el punto generador de orden n .

Firma: dado un mensaje m , se elige $k \leftarrow R[1, n-1]$, se calcula $(x_1, y_1) = k \cdot G$, $r = x_1 \bmod n$, $s = k^{-1}(H(m) + dr) \bmod n$. La firma es el par (r, s) .

Verificación: con la clave pública Q , se calcula $u_1 = s^{-1}H(m) \bmod n$, $u_2 = s^{-1}r \bmod n$, $(x_1, y_1) = u_1G + u_2Q$. La firma es válida si $r \equiv x_1 \pmod{n}$.

5.2 Justificación de EC frente a RSA para firmas

Característica	RSA-2048	EC P-256
Tamaño de clave	2 048 bits	256 bits
Tamaño de firma	256 bytes	64 a 72 bytes
Nivel de seguridad	Alrededor de 112 bits	Alrededor de 128 bits
Velocidad de firma	Lenta	Muy rápida
Almacenamiento en DB	Mayor	Menor

Dado que el sistema genera una firma ECDSA por cada aprobación de migrante, rectificación y cancelación ARCO, la eficiencia de la curva elíptica es relevante. RSA se reserva para la CA, que firma en menos ocasiones.

5.3 Generación y almacenamiento de claves

```
def generar_claves_ec():
    privada = ec.generate_private_key(ec.SECP256R1())
    priv_pem = privada.private_bytes(
        serialization.Encoding.PEM,
        serialization.PrivateFormat.PKCS8,
        serialization.NoEncryption()
    ).decode()
    pub_pem = privada.public_key().public_bytes(
        serialization.Encoding.PEM,
        serialization.PublicFormat.SubjectPublicKeyInfo
    ).decode()
    return priv_pem, pub_pem
```

5.4 Resolución de claves por rol

Fuente	Roles	Dónde vive
Sesión Flask (extraída del .p12)	admin, coord	Memoria; destruida al cerrar sesión
Campo ec_private_key DB	op	MySQL, persistente
Generada on-demand	op sin clave previa	Se genera y guarda en MySQL

La función `_obtener_o_crear_claves_ec(db, usuario)` encapsula esta lógica: primero consulta la sesión, luego la base de datos, y si no existe clave la genera y persiste.

5.5 Firma y verificación

```
def firmar_mensaje(priv_pem: str, mensaje: str) -> str:
    # Para admin/coord: prioriza la clave del .p12 en sesión
    llave_pem = session.get('cert_private_key') or priv_pem
    privada = serialization.load_pem_private_key(llave_pem.encode(), password=None)
    sig = privada.sign(mensaje.encode('utf-8'), ec.ECDSA(hashes.SHA256()))
    return base64.b64encode(sig).decode()

def verificar_firma(pub_pem: str, mensaje: str, firma_b64: str) -> bool:
    try:
        publica = serialization.load_pem_public_key(pub_pem.encode())
        publica.verify(
            base64.b64decode(firma_b64),
            mensaje.encode('utf-8'),
            ec.ECDSA(hashes.SHA256())
        )
        return True
    except Exception:
        return False
```

El resultado de `firmar_mensaje` es una firma DER codificada en Base64, de aproximadamente 88–96 caracteres. Cualquier alteración en el mensaje o en la firma produce una excepción en `verify()`, lo que garantiza la integridad de los datos almacenados.

5.6 Formato de los mensajes firmados

Operación	Formato del mensaje
Registro validado por OP	REGISTRO_OP id nombre fnac pais enviado_por op ts
Registro aprobado por Coord	REGISTRO_COORD id nombre fnac pais op coord ts ancla_op
Registro aprobado por Admin	REGISTRO_ADMIN id nombre fnac pais op admin ts ancla_op
Cancelación directa (coord)	CANCELACION mid folio coord ts motivo
Cancelación escalada (op→coord)	CANCELACION mid folio op coord ts motivo
Solicitud de eliminación	SOLICITUD_ELIMINACION mid folio coord ts motivo

6. Roles y Permisos

6.1 Modelo de roles

El sistema implementa control de acceso basado en roles (RBAC) con cuatro niveles. Cada ruta valida el rol activo mediante el decorador `@verificar_certificado` y, cuando corresponde, mediante comparación explícita contra `g.rol`.

Capacidad	Voluntario	Op	Coord	Admin
Crear solicitud de registro	✓	✓	✓	✓
Validar tickets (operativo)		✓		
Aprobar registros (coord)			✓	
Aprobar registros (override)				✓
Consultar padrón		✓	✓	✓
Editar datos de migrante			✓	✓
Solicitar cancelación ARCO		✓	✓	
Firmar cancelación (ECDSA)			✓	
Aprobar eliminación definitiva				✓
Emitir y revocar certificados				✓
Consultar bitácora de auditoría				✓
Gestionar cuentas				✓
Descargar Excel (padrón)		✓	✓	✓
Autenticación 2FA (.p12)			✓	✓

6.2 Flujos de inicio de sesión

Operativo y Voluntario:

POST /login → verificar credenciales → verificar cert en DB → sesión activa

Administrador y Coordinador:

POST /login → credenciales → session['pending_user']
 → GET /login_usb → POST /login_usb
 → verificar .p12 + email → cargar clave EC en sesión → sesión completa

6.3 Descripción de los roles

Voluntario.

Captura la información inicial de las personas que llegan al albergue. Su acción crea un ticket en estado pendiente_op que espera validación del operativo. No tiene acceso al padrón consolidado. Autenticación: contraseña.

Operativo.

Valida o rechaza los tickets de voluntarios. Puede consultar el padrón, emitir el derecho de Acceso ARCO y solicitar Rectificación y Cancelación (que requieren aprobación del coordinador).

Autenticación: contraseña.

Coordinador.

Aprueba tickets de registro, aplica rectificaciones ARCO de forma inmediata, firma solicitudes de cancelación con su clave EC y puede editar datos de migrantes. Su certificado USB es indelegable: el sistema verifica que el email del .p12 coincida con el usuario en sesión. Autenticación: contraseña + .p12.

Administrador.

Control total del sistema: emite y revoca certificados, gestiona cuentas, aprueba eliminaciones de registros verificando la firma ECDSA del coordinador (tanto para eliminaciones por causas administrativas como por derecho ARCO), y tiene acceso a la bitácora completa. Autenticación: contraseña + .p12.

6.4 Protección contra fuerza bruta

Tras diez intentos fallidos consecutivos, la cuenta se bloquea automáticamente:

```
UPDATE usuarios
SET intentos_fallidos = COALESCE(intentos_fallidos, 0) + 1,
    bloqueado = CASE
        WHEN COALESCE(intentos_fallidos, 0) + 1 >= 10 THEN 1
        ELSE COALESCE(bloqueado, 0)
    END
WHERE usuario = %s
```

Solo el administrador puede desbloquear la cuenta desde el panel de gestión de colaboradores.

7. Flujo de Registro de Migrantes

7.1 Flujo multi-nivel

El registro de un migrante sigue un flujo de aprobación cuyo recorrido depende del rol del capturista. La tabla solicitudes_registro_migrante actúa como cola de trabajo, y cada transición de estado va acompañada de una firma ECDSA del actor que la ejecuta.

Origen	Estados	Actores que firman
Voluntario	pendiente_op → pendiente_coord → aprobada	Operativo + Coordinador/Admin
Operativo directo	pendiente_coord → aprobada	Operativo (al crear) + Coordinador
Coordinador / Admin directo	Inserción inmediata	Coordinador o Admin
Admin override	Cualquier pendiente → aprobada	Admin

7.2 Aprobación masiva

Los endpoints /api/registro_migrante/bulk_coord_resolver y /api/registro_migrante/bulk_admin_resolver permiten aprobar o rechazar múltiples tickets en una sola petición. Para cada ticket, el sistema genera una firma ECDSA independiente, garantizando que el registro sea individualmente verificable aunque haya sido procesado en lote.

8. Derechos ARCO

8.1 Marco legal

La Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) reconoce cuatro derechos al titular de los datos. El sistema los implementa como flujos de trabajo verificables con registro en bitácora.

Derecho	Descripción	Implementación
Acceso	El titular puede consultar sus datos.	Operativo consulta /api/arco/acceso/<id>; cada consulta queda en bitácora como arco_acceso.
Rectificación	El titular puede corregir datos incorrectos.	Operativo propone cambios en solicitudes_arco_rect; coordinador aprueba o rechaza.
Cancelación	El titular puede solicitar la eliminación de sus datos.	Operativo solicita → coordinador firma con ECDSA → administrador verifica firma y elimina.
Oposición	El titular puede vetar usos específicos.	Flags de oposición bloquean usos concretos sin eliminar el registro.

8.2. Acceso

El derecho de Acceso permite a un migrante registrado solicitar consulta de todos los datos que el sistema conserva sobre él. El flujo no requiere firma criptográfica porque no modifica ni elimina información, pero sí exige autenticación de dos actores:

El operativo recibe la solicitud e identifica el registro correspondiente.

El coordinador autoriza la consulta y genera el reporte de datos desde el panel de coordinador.

El sistema registra el evento en log_certificados con el usuario solicitante, el rol autorizante y la marca temporal.

La consulta es de solo lectura; no altera el estado de ninguna tabla activa.

8.3. Rectificación

La rectificación corrige datos incorrectos o desactualizados en el registro de un migrante. A diferencia del acceso, esta operación sí modifica la base de datos y por tanto requiere firma:

El operativo envía la solicitud de cambio indicando el campo, el valor actual y el valor propuesto, que quedan almacenados en solicitudes_arco_rect.

El coordinador revisa la justificación y, si procede, firma el mensaje de rectificación con su clave EC P-256 del .p12.

El administrador verifica la firma con la clave pública del coordinador. Solo si la firma es válida se aplica el UPDATE sobre la tabla migrantes y se registra el cambio en log_certificados con el formato campo: "valor anterior" → "valor nuevo".

```
pythonvalida = verificar_firma(sol['coord_pubkey'],
                               sol['mensaje_firmado'],
                               sol['firma_coord'])
if valida:
    db.execute('UPDATE migrantes SET %s=%s WHERE id=%s',
              (campo, valor_nuevo, mid))
    # registra en log_certificados
```

8.4. Cancelación

La cancelación es la operación de mayor consecuencia: elimina permanentemente el registro. El flujo de tres niveles garantiza que ningún actor pueda ejecutarla unilateralmente:

El operativo solicita la cancelación con motivo.

El coordinador revisa y, si procede, firma el mensaje de cancelación con su clave EC P-256 del .p12.

El administrador verifica la firma con la clave pública del coordinador. Solo si la firma es válida se procede a eliminar el registro y archivar la operación en historial_eliminaciones.

```
pythonvalida = verificar_firma(sol['coord_pubkey'],
                             sol['mensaje_firmado'],
                             sol['firma_coord'])
if valida:
    db.execute('DELETE FROM migrantes WHERE id=%s', (mid,))
    # archiva en historial_eliminaciones
```

8.5. Oposición

El derecho de Oposición permite a un migrante manifestar que no consiente el uso de sus datos para una finalidad específica, sin que esto implique necesariamente su eliminación. El sistema lo gestiona como una anotación sobre el registro:

El operativo registra la solicitud de oposición con la finalidad objetada, que queda almacenada en solicitudes_arco_rect con tipo oposicion.

El coordinador evalúa la solicitud. Si procede, firma el mensaje de oposición con su clave EC P-256 del .p12 para dejar constancia de la decisión tomada.

El administrador verifica la firma y aplica la restricción correspondiente sobre el registro: una marca que limita el uso del dato objetado sin eliminarlo. El evento queda registrado en log_certificados.

La oposición es el único derecho ARCO cuyo resultado puede ser una negativa justificada; en ese caso el coordinador documenta la razón y el sistema registra igualmente la resolución con firma, garantizando trazabilidad independientemente del desenlace.

9. Rutas y API

9.1 Rutas de autenticación

Ruta	Método	Descripción
/	GET	Redirige al panel según rol
/login	GET/POST	Formulario de login (email + contraseña)
/login_usb	GET/POST	Verificación del .p12 (admin/coord)
/cambiar_pwd	GET/POST	Cambio obligatorio de contraseña
/logout	GET	Cierre de sesión
/solicitar_reset_pwd	POST	Solicitud de restablecimiento de contraseña

9.2 Rutas del panel de administración

Ruta	Método	Descripción
/admin	GET	Panel principal
/admin/api/metricas	GET	Conteo de certificados por estado
/admin/api/inicio_stats	GET	Estadísticas del dashboard
/admin/api/certificados	GET	Lista de certificados
/admin/api/log	GET	Últimos 50 eventos de auditoría
/admin/api/usuarios	GET	Lista de colaboradores
/admin/nuevo_voluntario	POST	Alta de colaborador
/admin/entregar_certificado	GET	Pantalla de entrega del .p12
/admin/descargar_p12	GET	ZIP con .p12 + .crt + README
/admin/revocar/<id>	POST	Revocar certificado
/admin/certificados/<id>/renovar	POST	Renovar certificado
/admin/certificados/<id>/eliminar	POST	Eliminar registro de certificado
/admin/api/solicitudes	GET	Solicitudes de eliminación pendientes
/admin/api/solicitudes/<id>/resolver	POST	Aprobar/rechazar eliminación
/admin/api/solicitudes_pwd	GET	Solicitudes de reset de contraseña
/admin/descargar/migrantes	GET	Excel del padrón (con filtros)
/admin/descargar/log	GET	Excel de la bitácora
/admin/descargar/eliminados	GET	Excel del historial de eliminados

9.3 Rutas de gestión de migrantes

Ruta	Método	Descripción
/api/migrantes	GET	Lista del padrón
/api/migrantes	POST	Crear solicitud/registro
/api/migrante/<id>	GET	Detalle de un registro
/api/migrantes/<id>	PUT	Editar datos
/api/migrantes/<id>	DELETE	Eliminación directa (admin)
/api/registro_migrante/pendientes_op	GET	Tickets pendientes para operativo
/api/registro_migrante/pendientes_coord	GET	Tickets pendientes para coordinador
/api/registro_migrante/<id>/op_validar	POST	Operativo valida/rechaza
/api/registro_migrante/<id>/coord_resolver	POST	Coordinador aprueba/rechaza

Ruta	Método	Descripción
/api/registro_migrante/bulk_coord_resolver	POST	Aprobación/rechazo masivo (coord)
/api/registro_migrante/bulk_admin_resolver	POST	Aprobación/rechazo masivo (admin)
/api/registro_migrante/mis_solicitudes	GET	Mis tickets enviados

9.4 Rutas ARCO

Ruta	Método	Descripción
/api/arco/buscar	GET	Búsqueda de migrantes
/api/arco/acceso/<id>	GET	Derecho de Acceso
/api/arco/rectificacion	POST	Solicitar/aplicar Rectificación
/api/arco/solicitudes_rect	GET	Lista de solicitudes de rectificación
/api/arco/solicitudes_rect/<id>/resolver	POST	Aprobar/rechazar rectificación
/api/arco/cancelacion/<id>	POST	Cancelación directa (coordinador, firmada)
/api/arco/cancelacion_op	POST	Solicitud de cancelación (operativo)
/api/arco/solicitudes_cancelacion_op	GET	Solicitudes pendientes para coordinador
/api/arco/solicitudes_cancelacion_op/<id>/resolver	POST	Coordinador aprueba/rechaza
/api/arco/verificar_firma/<id>	GET	Verificar firma ECDSA de solicitud
/api/arco/historial_completo	GET	Historial unificado ARCO

10. Bitácora de Auditoría

10.1 Diseño

La bitácora fue diseñada desde el inicio del proyecto, no añadida posteriormente. Esto permite que el cumplimiento de la LFPDPPP no haya requerido más trabajo, cada evento relevante se registra en el momento en que ocurre, con la dirección IP del cliente y un campo de detalle de texto libre.

```
def log_evento(resultado, usuario=None, rol=None, serial=None, detalle=None):
    db = obtener_db()
    _exec(db,
        'INSERT INTO log_certificados '
        '(serial, usuario, rol, ip_origen, resultado, detalle)'
        'VALUES (%s, %s, %s, %s, %s, %s)',
        (serial, usuario, rol, request.remote_addr, resultado, detalle)
    )
    db.commit()
```

El campo detalle registra información contextual como cambios de campo en el formato: campo: "valor_anterior" → "valor_nuevo".

10.2 Catálogo de eventos

Código	Descripción
login_exitoso	Autenticación completada
login_fallido	Credenciales incorrectas
cert_revocado	Intento de acceso con certificado revocado
cert_expirado	Intento de acceso con certificado vencido
cert_emitido	Nuevo certificado generado
cert_revocado_admin	Certificado revocado manualmente por admin
rol_modificado	Cambio de rol o datos de colaborador
pwd_cambiada	Cambio de contraseña
acceso_denegado	Intento de acceso no autorizado
migrante_registrado	Inserción directa en el padrón
migrante_actualizado	Edición de datos de un migrante
migrante_eliminado	Eliminación definitiva aprobada
arco_acceso	Ejercicio del derecho de Acceso ARCO
arco_rect_solicitada	Operativo solicita Rectificación
arco_rect_aprobada	Coordinador aprueba Rectificación
arco_rect_rechazada	Coordinador rechaza Rectificación
arco_cancel_coord_firmada	Coordinador firma solicitud de Cancelación
arco_cancel_coord_rechazada	Coordinador rechaza solicitud de Cancelación
registro_mig_enviado_voluntario	Voluntario envía ticket de registro

Código	Descripción
registro_mig_op_validado	Operativo valida ticket
registro_mig_op_rechazado	Operativo rechaza ticket
registro_mig_coord_aprobado	Coordinador aprueba ticket
registro_mig_coord_rechazado	Coordinador rechaza ticket
registro_mig_admin_aprobado	Admin aprueba ticket (override)
registro_mig_insertado	Migrante insertado en el padrón definitivo
pwd_reset_solicitado	Colaborador solicita reset de contraseña
pwd_reset_aprobado	Admin aprueba reset
pwd_reset_rechazado	Admin rechaza solicitud de reset

10.3 Exportación

Las rutas /admin/descargar/log y /admin/descargar/migrantes generan libros Excel con openpyxl, con cabeceras formateadas en el color corporativo y soporte para filtros por rango de fechas, país de origen y grupo de vulnerabilidad.

11. Instalación y Configuración

11.1 Requisitos

- Python 3.12 o superior.
- MySQL 8.0 o superior.
- Una base de datos llamada casa_monarca creada previamente.

11.2 Dependencias

```
pip install flask flask-session cryptography werkzeug pymysql openpyxl
```

11.3 Variables de entorno

Variable	Valor por omisión	Descripción
DB_HOST	localhost	Host del servidor MySQL
DB_PORT	3306	Puerto MySQL
DB_USER	root	Usuario MySQL
DB_PASSWORD	(vacío)	Contraseña MySQL
DB_NAME	casa_monarca	Nombre de la base de datos
CA_PASSWORD	demo-ca-pwd-...	Contraseña para cifrar la clave de la CA
MTLS_ENABLED	1	1 = TLS; 0 = HTTP plano

11.4 Puesta en marcha

```
# 1. Crear la base de datos
mysql -u root -p -e "CREATE DATABASE casa_monarca CHARACTER SET utf8mb4;"
mysql -u root -p casa_monarca < db/schema_mysql.sql

# 2. Configurar variables (Windows PowerShell)
$env:DB_USER="root"; $env:DB_PASSWORD="contraseña"

# 3. Primer arranque: genera CA, cert de servidor, siembra DB
python app.py
```

Importante: CA_PASSWORD debe establecerse antes del primer arranque y no modificarse posteriormente. Cambiarla después de generar la CA obliga a regenerar todos los certificados.

11.5 Despliegue en producción

```
# Instalar gunicorn
pip install gunicorn
```

```
# Arrancar la aplicación
gunicorn -w 4 -b 127.0.0.1:8000 app:app

# Configuración nginx (fragmento)
# server {
#   listen 443 ssl;
#   ssl_certificate /ruta/ca/server.pem;
#   ssl_certificate_key /ruta/ca/server.key;
#   location / {
#     proxy_pass http://127.0.0.1:8000;
#   }
# }
```

12. Consideraciones de Seguridad y Trabajo Futuro

12.1 Resumen de decisiones de seguridad

Aspecto	Implementación
Contraseñas	PBKDF2-HMAC-SHA256 con salt (Werkzeug)
SQL injection	Parámetros %s en todas las consultas
Sesiones	Firmadas con SESSION_USE_SIGNER=True
Fuerza bruta	Bloqueo automático tras 10 intentos fallidos
Autenticación 2FA	Contraseña + certificado .p12 (admin y coord)
Clave privada .p12	Solo en memoria de sesión; nunca toca el disco
Integridad de ops	Firmas ECDSA P-256 verificables en cualquier momento
Revocación	Serial verificado en DB en cada login con .p12

12.2 Limitaciones conocidas

- Heterogeneidad criptográfica: la CA usa RSA-4096 y los certificados de usuario contienen claves EC P-256. Esta combinación es funcional pero introduce heterogeneidad. La arquitectura más homogénea sería una CA EC P-384.
- Claves EC de operativos en la base de datos: la clave privada EC de los operativos se almacena en MySQL sin cifrado adicional. En un entorno de alta seguridad se recomendaría cifrarla con una clave derivada de la contraseña del usuario o utilizar un HSM.
- Sesiones en filesystem: el backend de sesiones es incompatible con despliegues de alta disponibilidad y se pierde al redespigar. Redis o una tabla de base de datos son alternativas adecuadas.
- Migraciones manuales: el sistema de migraciones mediante ALTER TABLE en try/except carece de versionado y rollback. Herramientas como Alembic o Flyway serían más apropiadas a largo plazo.
- Zona horaria fija: el offset UTC-6 no contempla el horario de verano de Monterrey (CDT, UTC-5, de marzo a noviembre). Se recomienda almacenar en UTC y convertir al presentar.

12.3 Hoja de ruta

Fase I — Producción:

Despliegue en Railway con nginx y gunicorn. Gestión de variables de entorno con secretos de la plataforma. TLS estable bajo carga.

Fase II — Operación:

Respallos automáticos de MySQL. Monitoreo de uptime. Rotación automática de certificados cinco días antes de su expiración, con notificación por correo al titular.

Fase III — Expansión:

Migración de la CA a EC P-384 para homogeneizar la arquitectura criptográfica. Cifrado de claves EC de operativos. Tableros de estadísticas para coordinación. Integración de autenticación con proveedor de identidad externo (OIDC).

Referencias

1. [1] Python Cryptographic Authority (PyCA). cryptography library. <https://cryptography.io/>
2. [2] NIST. FIPS 186-5: Digital Signature Standard (DSS). National Institute of Standards and Technology, 2023.
3. [3] NIST. SP 800-186: Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters. 2023.
4. [4] RFC 5480 — Elliptic Curve Cryptography Subject Public Key Information. IETF, 2009.
5. [5] RFC 7292 — PKCS #12: Personal Information Exchange Syntax v1.1. IETF, 2014.
6. [6] Diario Oficial de la Federación. Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP). México, 2010.
7. [7] Pallets Projects. Flask Web Framework. <https://flask.palletsprojects.com/>
8. [8] Oracle Corporation. MySQL 8.0 Reference Manual, 2024.
9. [9] Hankerson, D., Vanstone, S., Menezes, A. Guide to Elliptic Curve Cryptography. Springer, 2004.

Apéndice A. Glosario

Término	Definición
2FA	Autenticación de dos factores. Requiere dos elementos independientes para verificar la identidad: algo que se sabe (contraseña) y algo que se posee (certificado en USB).
ARCO	Acceso, Rectificación, Cancelación, Oposición. Derechos reconocidos al titular de datos personales por la LFPDPPP.
CA	Certificate Authority (Autoridad Certificadora). Entidad que firma certificados X.509 para garantizar su autenticidad.
ECDSA	Elliptic Curve Digital Signature Algorithm. Esquema de firma digital basado en criptografía de curva elíptica.
LFPDPPP	Ley Federal de Protección de Datos Personales en Posesión de los Particulares. Ley mexicana de privacidad de datos personales.
P-256	Curva elíptica estandarizada por NIST (SECP256R1 / prime256v1), definida sobre un campo primo de 256 bits. Utilizada para las firmas de operaciones en este sistema.
PBKDF2	Password-Based Key Derivation Function 2. Función de derivación de claves usada para almacenar contraseñas con resistencia a ataques de fuerza bruta.
PKI	Public Key Infrastructure. Conjunto de roles, políticas y procedimientos para gestionar certificados digitales.
PKCS#12	Estándar criptográfico que define un formato de archivo (.p12) para empaquetar una clave privada junto con su certificado X.509 y la cadena de CA.
RBAC	Role-Based Access Control. Modelo de control de acceso donde los permisos se asignan a roles y los usuarios se asignan a roles.
TLS	Transport Layer Security. Protocolo criptográfico para comunicaciones seguras en red.
X.509	Estándar de infraestructura de clave pública que define el formato de los certificados digitales.